24.	2	Instruction Formats and Addressing Modes of 8086.	CO 3/BT3	SB/LCD
25.	2	Instruction Set, Assembler Directives.	CO3 /BT3	SB/LCD
26.	2	Assembly Language Programs.	CO3 /BT4	SB/LCD
(10 Hours) UNIT-IV : Input-Output Organization and Memory Organization				
27	2	Input-OutputInterface, Asynchronous data transfer.	CO4 /BT2	SB/LCD
28	1	Modes of Transfer.	CO4 /BT3	SB/LCD
29	2	Priority Interrupt Direct memory Access.	CO4 /BT3	SB/LCD
30	2	Memory Hierarchy, Main Memory	CO4 /BT4	SB/LCD
31	2	Auxiliary memory, Associate Memory,	CO4 /BT4	SB/AI TOOLS
32	1	Cache Memory.	CO4 /BT3	SB/AI TOOLS
(10 Hours) UNIT-V : Pipeline and Vector Processing and Multi Processors				
33	2	Parallel Processing, Pipelining, Arithmetic Pipeline,	CO5 /BT3	SB/AI
34	2	Instruction Pipeline, RISC Pipeline, Vector Processing, Array Processor	CO5 /BT3	SB/AI
35	2	Characteristics of Multiprocessors, Interconnection Structures, Interprocessor arbitration,	CO5 /BT3	SB/AI
36	2	Interprocessor communication and synchronization	CO5 /BT4	SB/AI
37	2	Cache Coherence	CO5 /BT3	SB/AI

Total number of classes: 52

12. Detailed Lecture NOTES

Unit-I

Introduction, Register Transfer Language and Micro operations and Basic Structure of Computers

Digital Computer

The digital computer is a digital system that performs various computational tasks. Digital computers use the binary number system, which has two digits: 0 and 1. A binary digitis called a bit. Information is represented in digital computers in groups of bits. By using various coding techniques, groups of bits can be made to represent not only binary numbers but also other discrete symbols, such as decimal digits or letters of the alphabet. By judicious use of binary arrangements and by using various coding techniques, the groups of bits are used to develop complete sets of instructions for performing various types of computations.

A computer system is sometimes subdivided into two functional entities

1- The hardware of the computer consists of all the electronic components and electromechanical devices that comprise the physical entity of the device.

2- Computer software consists of the instructions and data that the computer manipulatesto perform various data-processing tasks.

The system software of a computer consists of a collection of programs whose purpose is to make more effective use of the computer. The programs included in a systems software package are referred to as the operating system.

Computer Hardware

The hardware of the computer is usually divided into three major parts, as shown in Fig(1)

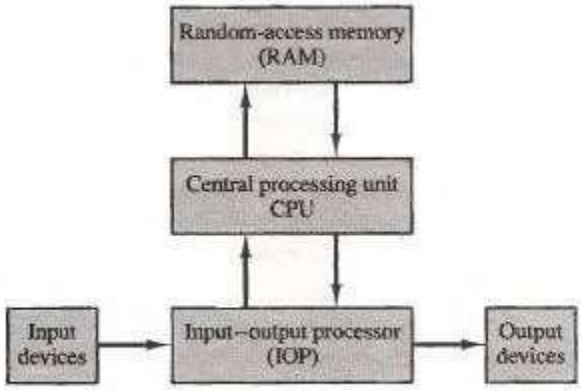


Figure 1 Block diagram of a digital computer.

The central processing unit (CPU) contains arithmetic and logic unit for manipulating data, a number of registers for storing data, and control circuits for fetching and executing instructions. The memory of a computer contains storage for instructions and data. It is calleda random- access memory (RAM) because the CPU can access any location in memory at random and retrieve the binary information within a fixed interval of time. The input and output processor (IOP) contains electronic circuits for communicating and controlling the transfer of information between the computer and the outside world. The input and output devices connected to the computer include keyboards, printers, terminals, magnetic disk drives, and other communication devices.

Computer Organization

Computer organization is concerned with the way the hardware components operate andthe way they are connected together to form the computer system. The various components are assumed to be in place and the task is to investigate the organizational structure to verify that the computer parts operate as intended.

Computer Design

Computer design is concerned with the hardware design of the computer. Once the computer specifications are formulated, it is the task of the designer to develop hardware for the system. Computer design is concerned with the determination of what hardware should be used and how the parts should be connected. This aspect of computer hardware is sometimes referred to as computer implementation.

Computer Architecture

Computer architecture is concerned with the structure and behavior of the computer as seen by the

user. It includes the information formats, the instruction set, and techniques for addressing memory. The architectural design of a computer system is concerned with the specifications of the various functional modules, such as processors and memories, and structuring them together into a computer system.

Register Transfer Language

Digital systems vary in size and complexity from a few integrated circuits to a complex of interconnected and interacting digital computers. Digital system design invariably uses a modular approach. The modules are constructed from such digital components as registers, decoders, arithmetic elements, and control logic. The various modules are interconnected with common data and control paths to form a digital computer system. Digital modules are best defined by the registers they contain and the operations that are performed on the data stored in them. The operations executed on data stored in registers are called microoperations. A microoperation is an elementary operation performed on the informationstored in one or more registers. The result of the operation may replace the previous binary informationof a register or may betran

The internal hardware organization of a digital computer is best defined by specifying:

1- The set of registers it contains and their function.
2- The sequence of microoperations performed on the binary information stored in the registers.
3- The control that initiates the sequence of microoperations.

The symbolic notation used to describe the microoperation transfers among registers is called a register transfer language.

The term "register transfer" implies the availability of hardware logic circuits that can perform a stated microoperation and transfer the result of the operation to the same or another register.

The word "language" is borrowed from programmers, who apply this term to programming languages.

A register transfer language is a system for expressing in symbolic form the microoperation sequences among the registers of a digital module.

Register Transfer

Computer registers are designated by capital letters (sometimes followed by numerals) to denote the function of the register.

The representation of registers in block diagram form is shown in Fig(2):

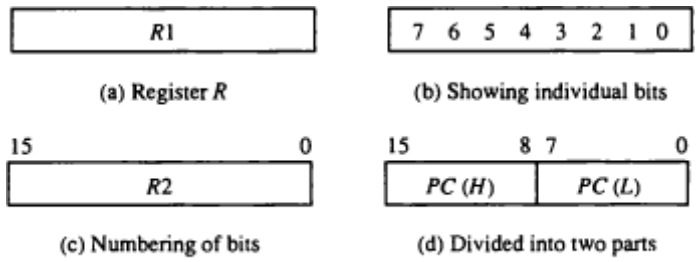


Figure 2 Block diagram of register.

a- Rectangular box with the name of the register inside.

b- The individual bits.

c- The numbering of bits in a 16-bit register can be marked on top of the box.

d- 16-bit register is partitioned into two parts. Bits 0 through 7 are assigned the symbol L (for low byte) and bits 8 through 15 are assigned the symbol H (for high byte). The name of the 16-bit register is PC. The symbol PC(0-7) or PC(L) refers to the low-order byte and PC(8-15) or PC(H) to the high-order byte.

Information transfer from one register to another is designated in symbolic form by meansof a replacement operator. The statement:

$$R2 \leftarrow R1$$

Denotes a transfer of the content of register Rl into register R2. It designates a replacement of the content of R2 by the content of Rl. By definition, the content of the source register Rl does not change after the transfer.

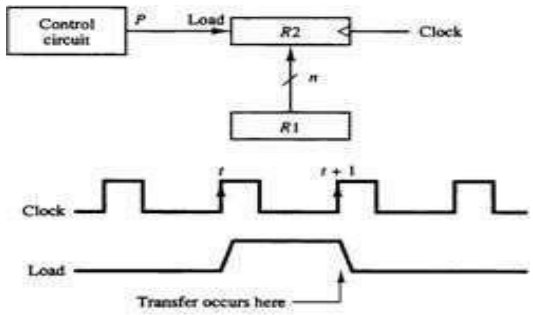
If we want the transfer to occur only under a predetermined control condition. This canbe shown by means of an if-then statement.

$$(P = 1)th(R2 \leftarrow R1)$$

where P is a control signal generated in the control section. It is sometimes convenient to separate the control variables from the register transfer operation control function byspecifying a control function.

$$P: R2 \leftarrow R1$$

The control condition is terminated with a colon. It symbolizes the requirement that the transfer operation be executed by the hardware only if P = 1.



Transfer from R1 to R2 when $P = 1$.

To separate two or more operations that is executed at the same time by using the comma asthe statement:

$$T: R2 \leftarrow R1, R5 \leftarrow R3$$

The basic symbols of the register transfer notation are listed in Table (1) Registers are denoted by capital letters, and numerals may follow the letters. Parentheses are used to denotea part of a register by specifying the range of bits or by giving a symbol name to a portion of a register.

TABLE 1 Basic Symbols for Register Transfers

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	*MAR*, *R2*
Parentheses ( )	Denotes a part of a register	*R2*(0–7), *R2*(*L*)
Arrow ←	Denotes transfer of information	*R2* ← *R1*
Comma ,	Separates two microoperations	*R2* ← *R1*, *R1* ← *R2*

Bus and Memory Transfers

A typical digital computer has many registers, and paths must be provided to transfer information from one register to another. The number of wires will be excessive if separate

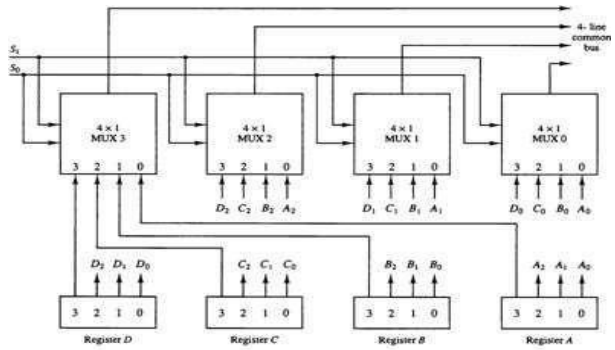


Fig 3 **Bus system for four registers**

lines are used between each register and all other registers in the system. A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time. Control signals determine which register is selected bythe bus during each particular register transfer. The multiplexers select the source registerwhose binary information is then placed on the bus. For example, the construction of a bussystem for four registers is shown in Fig(3) Each register has four bits, numbered 0 through 3. The bus consists of four 4x1 multiplexers each having four data inputs, 0 through 3, and twoselection inputs, S1 and S0.

The table(2) shows the register that is selected by the bus for each of the four possiblebinary values of the selection lines.

Table 2 Function for Bus of Fig

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

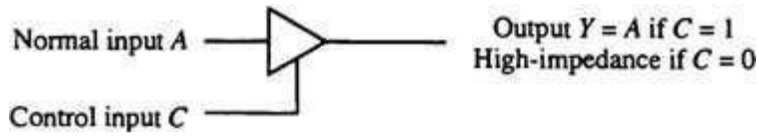
The symbolic statement for a bus transfer may mention the bus or its presence may be implied in the statement. When the bus is includes in the statement, the register transfer is symbolized as follows:

$$Bus \leftarrow \quad , \quad R1 \leftarrow Bus$$

The content of register C is placed on the bus, and the content of the bus is loaded into register Rl by activating its load control input. If the bus is known to exist in the system, it may be convenient just to show the direct transfer.

$$R1 \leftarrow C$$

A bus system can be constructed with three-state gates. The graphic symbol of a three- state buffer gate is shown:



Graphic symbols for three-state buffer.

To construct a common bus for four registers of n bits each using three-state buffers, we needn circuits with four buffers in each as shown in Fig(4). Each group of four buffers receives one significant bit from the four registers.

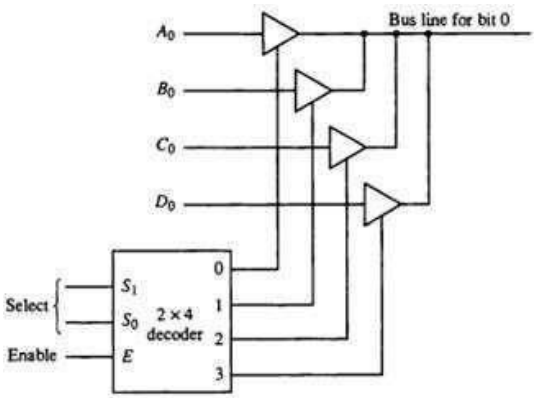


Figure 4 Bus line with three state-buffers.

The transfer of information from a memory word to the outside environment is called a read operation. The transfer of new information to be stored into the memory is called a write operation. Consider a memory unit that receives the address from a register, called the address register, symbolized by AR. The data are transferred to another register, called the data register, symbolized by DR.

$$Read: DR \leftarrow M[AR]$$

The write operation transfers the content of a data register to a memory word M selected bythe address.

$$Write: M[AR] \leftarrow DR$$

Arithmetic Microoperations

The arithmetic operations are listed in the Table(3):

TABLE 3 Arithmetic Microoperations

Symbolic designation	Description
$R3 \leftarrow R1 + R2$	Contents of $R1$ plus $R2$ transferred to $R3$
$R3 \leftarrow R1 - R2$	Contents of $R1$ minus $R2$ transferred to $R3$
$R2 \leftarrow \overline{R2}$	Complement the contents of $R2$ (1's complement)
$R2 \leftarrow \overline{R2} + 1$	2's complement the contents of $R2$ (negate)
$R3 \leftarrow R1 + \overline{R2} + 1$	$R1$ plus the 2's complement of $R2$ (subtraction)
$R1 \leftarrow R1 + 1$	Increment the contents of $R1$ by one
$R1 \leftarrow R1 - 1$	Decrement the contents of $R1$ by one

The multiply and divide are not listed in Table(3), these two operations are valid arithmetic operations but are not included in the basic set of microoperations. In most computers, the multiplication operation is implemented with a sequence of add and shift microoperations. Division is implemented

with a sequence of subtract and shift microoperations.
The digital circuit that generates the arithmetic sum of two binary numbers of any length is called a binary adder as shown in Fig(5).

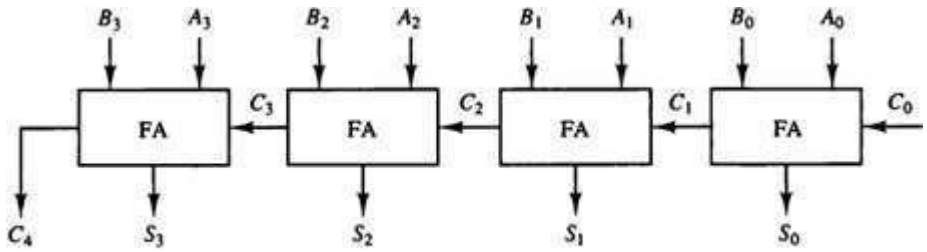


Figure 5 4-bit binary adder.

The addition and subtraction operations can be combined into one common circuit by including an exclusive-OR gate with each full-adder as shown in Fig(6).

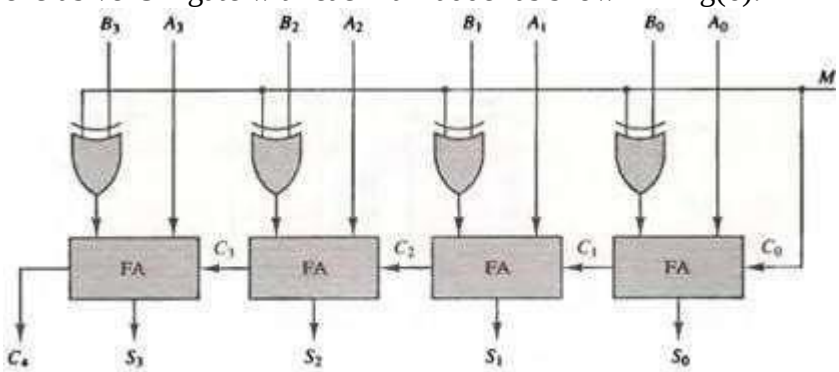


Figure 6 4-bit adder-subtractor.

The increment microoperation adds one to a number in a register. For example, if a 4-bit register has a binary value 0110, it will go to 0111 after it is incremented. The diagram of a 4- bit combinational circuit incrementer is shown in Fig(7):

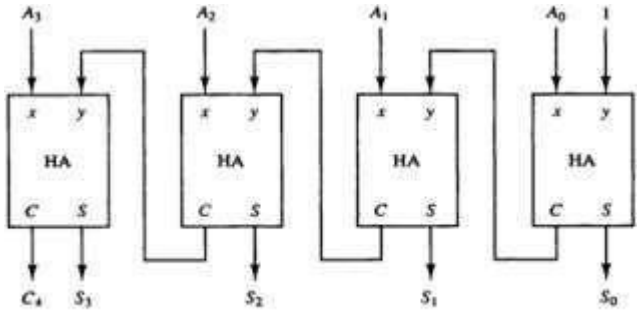


Figure 7 4-bit binary incrementer.

The arithmetic microoperations listed in the Table above can be implemented in one composite arithmetic circuit. The basic component of an arithmetic circuit is the parallel adder.By controlling the data inputs to the adder, it is possible to obtain different types of arithmetic operations as shown in Fig(8).

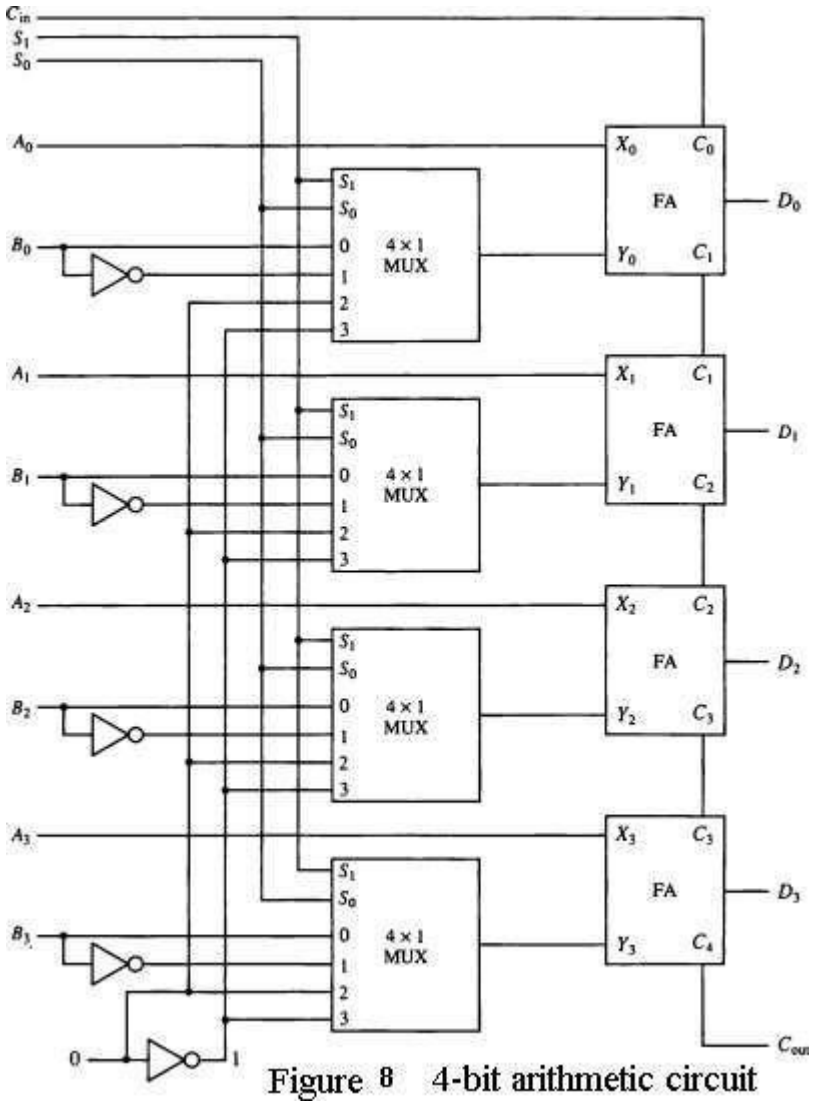


Figure 8 4-bit arithmetic circuit

It is possible to generate the eight arithmetic microoperations listed in Table(4):

TABLE 4 Arithmetic Circuit Function Table

Select S_1	Select S_0	Select C_{in}	Input Y	Output $D = A + Y + C_{in}$	Microoperation
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\overline{B}$	$D = A + \overline{B}$	Subtract with borrow
0	1	1	$\overline{B}$	$D = A + \overline{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

Logic Microoperations

Logic microoperations specify binary operations for strings of bits stored in registers. These operations consider each bit of the register separately and treat them as binary variables. For example, the exclusive-OR microoperation with the contents of two registers Rl and R2 is symbolized by the statement:

$$P: R1 \leftarrow R1 \oplus R2$$

It specifies a logic microoperation to be executed on the individual bits of the registers provided that the control variable P = 1. As a numerical example, assume that each register has four bits. Let the content of Rl be 1010 and the content of R2 be 1100. The exclusive-OR microoperation stated above symbolizes the following logic computation:

1010	Content of $R1$
1100	Content of $R2$
0110	Content of $R1$ after $P = 1$

There are 16 different logic operations that can be performed with two binary variables. They can be determined from all possible truth tables obtained with two binary variables as shown in Table(5):

TABLE 5 Truth Tables for 16 Functions of Two Variables

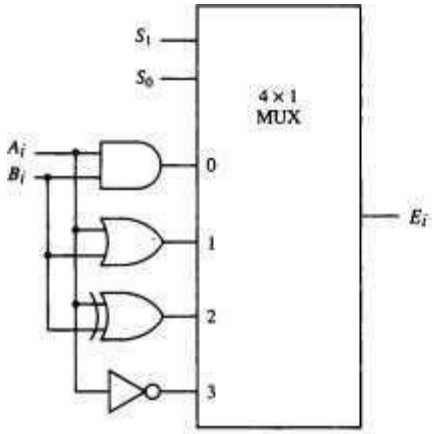
x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

The 16 Boolean functions of two variables x and y are expressed in algebraic form in the first column of Table(6):

TABLE 6 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow$ all 1's	Set to all 1's

The diagram shows (Fig 9-a) one typical stage with subscript i. For a logic circuit with n bits, the diagram must be repeated n times for i = 0,1, 2,..., n - 1. The selection variables are applied to all stages. The function table in Fig.(9-b) lists the logic microoperations obtained for each combination of the selection variables.



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

Figure 9 One stage of logic circuit.

Shift Microoperations

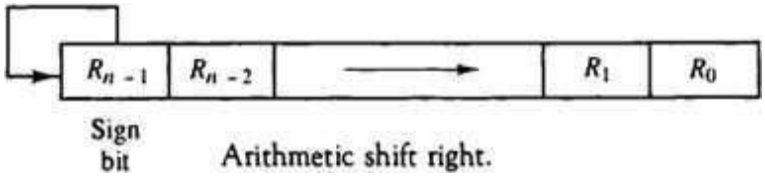
Shift microoperations are used for serial transfer of data. The contents of a register can be shifted to the left or the right. There are three types of shifts: logical, circular, and arithmetic. The symbolic notation for the shift microoperations is shown in Table (7):

TABLE 7 Shift Microoperations

Symbolic designation	Description
$R \leftarrow$ shl R	Shift-left register R
$R \leftarrow$ shr R	Shift-right register R
$R \leftarrow$ cil R	Circular shift-left register R
$R \leftarrow$ cir R	Circular shift-right register R
$R \leftarrow$ ashl R	Arithmetic shift-left R
$R \leftarrow$ ashr R	Arithmetic shift-right R

An arithmetic shift is a microoperation that shifts a signed binary number to the left or right. The arithmetic shift-left inserts a 0 into R_0, and shifts all other bits to the left. The initial bit of R_{n-1} is lost and replaced by the bit from R_{n-2} . A sign reversal occurs if the bit in R_{n-1} changes in value after the shift and caused an overflow.

The arithmetic shift-right leaves the sign bit unchanged and shifts the number (including the sign bit) to the right.



Arithmetic shift right.

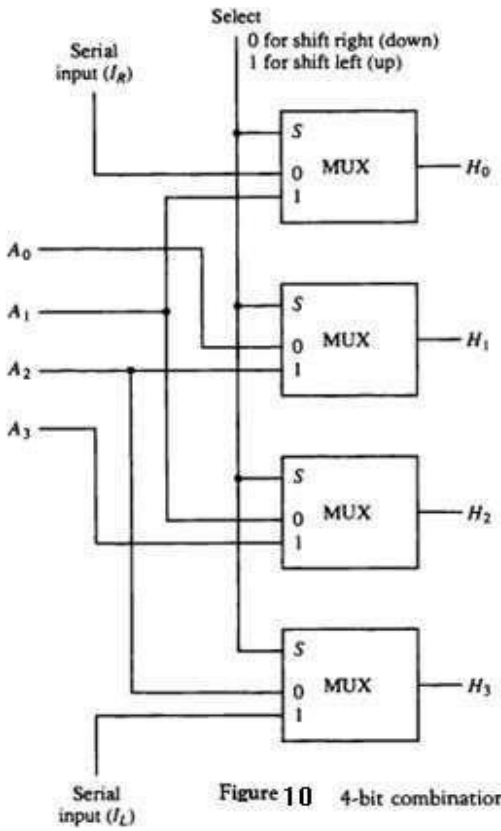
Ex: If the content of 8 bits register is (10100011). What is the result of the operation after executing to the register:

a. shl R: shift left register by 3. b. cil R : circular shift left register by 3.

c. ashl R: arithmetic shift left register by 3. d. ashr R: arithmetic shift right register by 3.Ans:

(a) 00011000. (b) 00011101. (c) 00011000. Overflow (d) 11110100

A combinational circuit shifter can be constructed with multiplexers as shown in Fig(10). The 4-bit shifter has four data inputs, A0 through A3, and four data outputs, H0through H3. There are two serial inputs, one for shift left (IL) and the other for shift right (IR).



Function table

Select	Output			
S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

Figure 10 4-bit combinational circuit shifter.

Arithmetic Logic Shift Unit

Computer systems employ a number of storage registers connected to a common operational unit called an arithmetic logic unit, aSBreviated ALU. The arithmetic, logic, and shift circuits introduced in previous sections can be combined into one ALU with common selection variables. One stage of an arithmetic logic shift unit is shown in Fig(11) with the functional table(8):

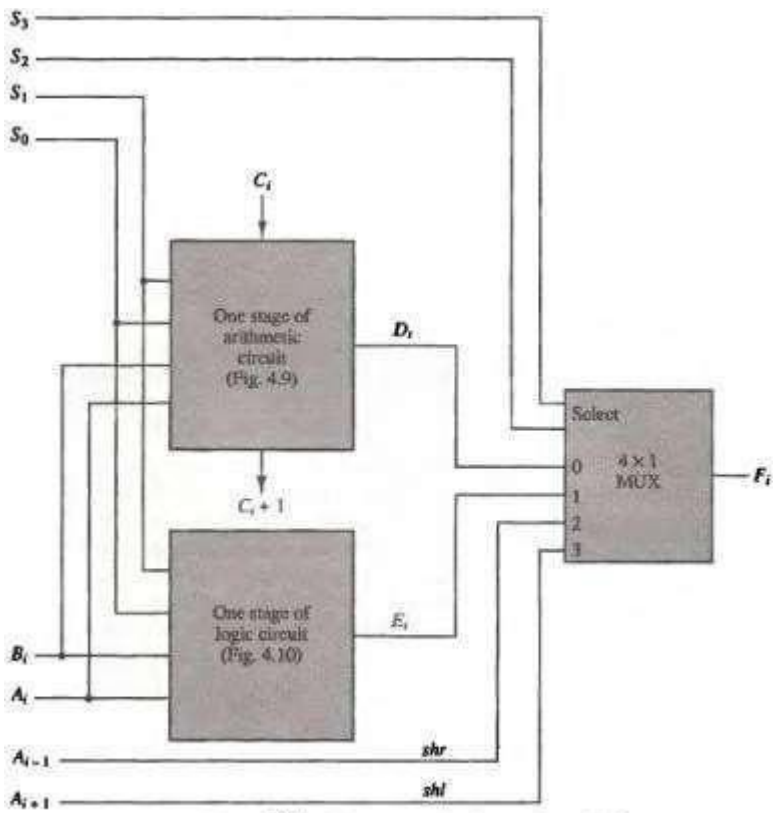


Figure 11 One stage of arithmetic logic shift unit.

TABLE 8 Function Table for Arithmetic Logic Shift Unit

Operation select						
S_3	S_2	S_1	S_0	C_{in}	Operation	Function
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	×	$F = A \wedge B$	AND
0	1	0	1	×	$F = A \vee B$	OR
0	1	1	0	×	$F = A \oplus B$	XOR
0	1	1	1	×	$F = \overline{A}$	Complement A
1	0	×	×	×	$F = \text{shr } A$	Shift right A into F
1	1	×	×	×	$F = \text{shl } A$	Shift left A into F

Basic Structure of Computers

Computer Architecture in general covers three aspects of computer design namely: Computer Hardware, Instruction set Architecture and Computer Organization.

Computer hardware consists of electronic circuits, displays, magnetic and optical storage media and communication facilities.

Instruction set Architecture is programmer visible machine interface such as instruction set, registers, memory organization and exception handling. Two main approaches are mainly CISC (Complex Instruction Set Computer) and RISC (Reduced Instruction Set Computer)

Computer Organization includes the high level aspects of a design, such as memory system, the bus structure and the design of the internal CPU.

Instruction Codes

A computer instruction is a binary code that specifies a sequence of microoperations for the computer. Instruction codes together with data are stored in memory. The computer reads each instruction from memory and places it in a control register. The control then interprets thebinary code of the instruction and proceeds to execute it by issuing a sequence of micro operations. Every computer has its own unique instruction set. The ability to store and execute instructions, the stored program concept, is the most important property of a general- purpose computer. An instruction code is a group of bits that instruct the computer to perform a specific operation. It is usually divided into parts, each having its own particular interpretation. The most basic part of an instruction code is its operation part. The operation code of an instruction is a group of bits that define such operations as add, subtract, multiply, shift, and complement. The number of bits required for the operation code of an instruction depends on the total number of operations available in the computer.

The simplest way to organize a computer is to have one processor register and an instruction code format with two parts. The first part specifies the operation to be performed and the second specifies an address. The memory address tells the control where to find an operand in memory. This operand is read from memory and used as the data to be operated on together with the data stored in the processor register.

For a memory unit with 4096 words we need 12 bits to specify an address since 2^{12}= 4096. If we store each instruction code in one 16-bit memory word, we have available four bits for the operation code (aSBreviated opcode) to specify one out of

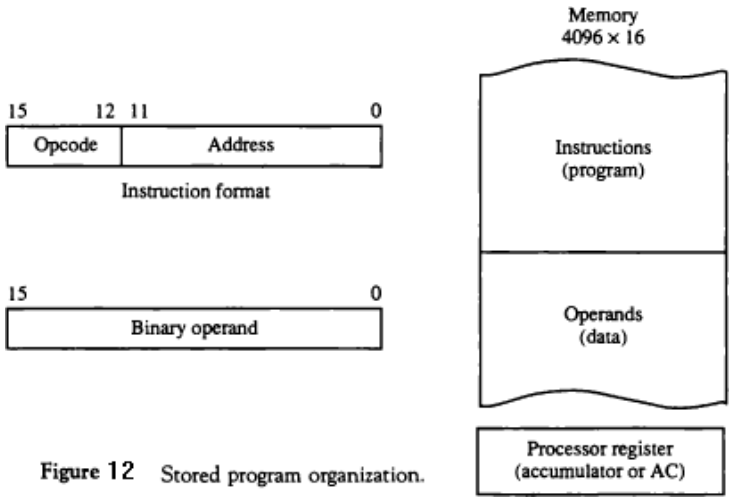


Figure 12 Stored program organization.

16 possibleoperations, and 12 bits to specify the address of an operand. The control reads a 16- bit instruction from the program portion of memory. It uses the 12-bit address part of the instruction to read a 16-bit operand from the data portion of memory. Fig(12) depicts this type of organization.

Computers that have a single-processor register usually assign to it the name accumulator and label it AC.

For example, operations such as clear AC, complement AC, and increment AC operate on data stored in the AC register.

When the second part of an instruction immediate code specifies an operand, this type is called **immediate operand**. When the second part specifies the address of an operand, the instruction is said to have a **direct address**. The third possibility called **indirect address**, where the bits in the second part of the instruction designate an address of a memory word in which the address of the operand is found. One bit of the instruction code can be used to distinguish between a direct and an indirect address.

As an illustration of this configuration, consider the instruction code format shown in Fig(13).

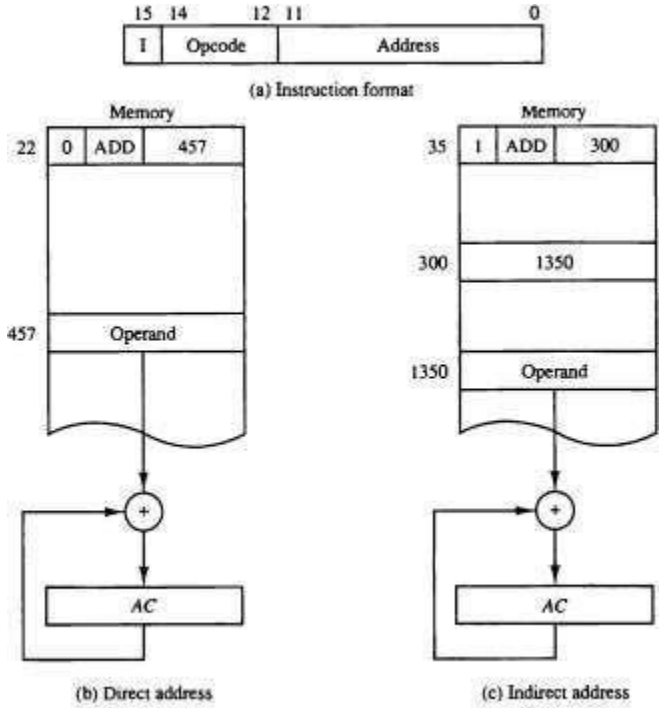


Fig 13 Demonstration of direct and indirect address.

Computer Registers

Computer instructions are normally stored in consecutive memory locations and are executed sequentially one at a time. The control reads an instruction from a specific address in memory and executes it. It then continues by reading the next instruction in sequence and executes it, and so on. This type of instruction sequencing needs a counter to calculate the

address of the next instruction after execution of the current instruction is completed. The computer needs processor registers for manipulating data and a register for holding a memory address. These requirements dictate the register configuration are listed in Table(9) and shown in Fig(14).

TABLE 9 List of Registers for the Basic Computer

Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

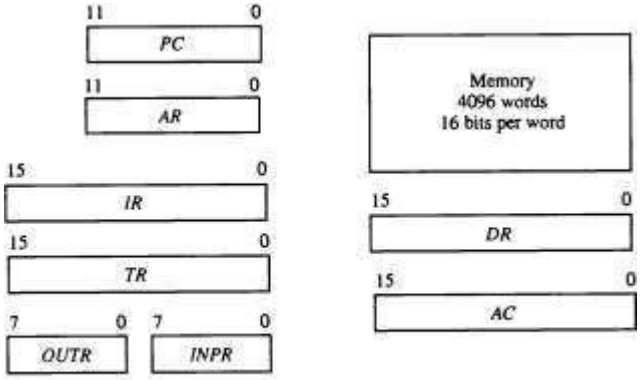


Figure 14 Basic computer registers and memory.

The basic computer has eight registers, a memory unit, and a control unit. The connection of the registers and memory of the basic computer to a common bus system is shown in Fig(15).

Two registers, AR and PC, have 12 bits each since they hold a memory address. When the contents of AR or PC are applied to the 16-bit common bus, the four most significant bits are set to 0's. When AR or PC receive information from the bus, only the 12 least significant bits are transferred into the register.

The input register INPR and the output register OUTR have 8 bits each and communicate with the eight least significant bits in the bus. The INPR receives a character from an input device which is then transferred to AC. OUTR receives a character from AC and delivers it to an output device. There is no transfer from OUTR to any of the other registers.

Five registers have three control inputs: LD (load), INR (increment), and CLR (clear).

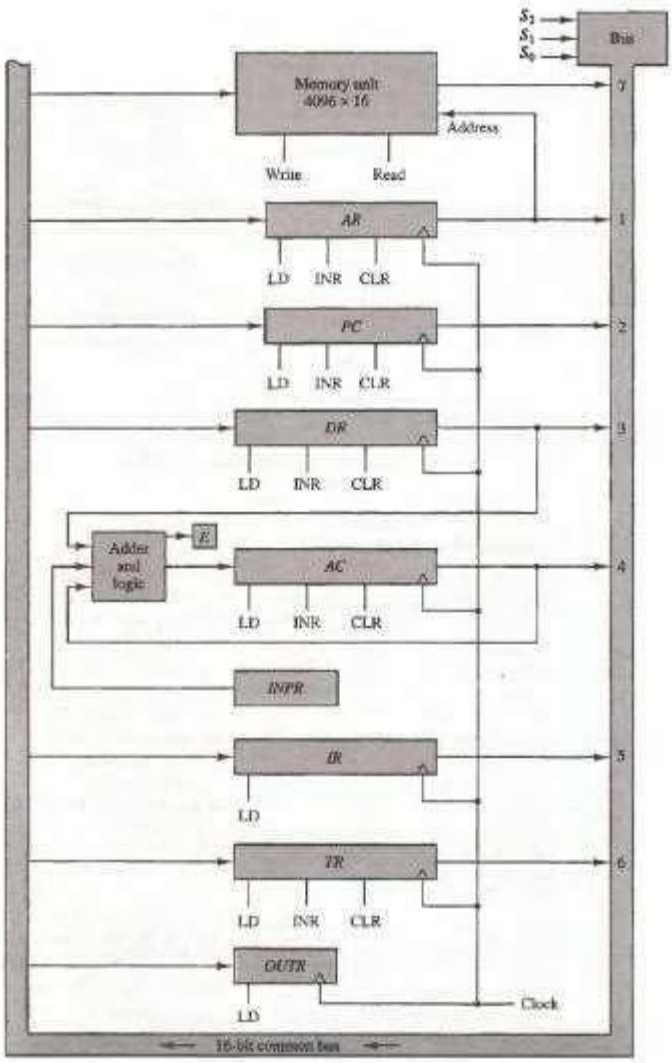


Figure 15 Basic computer registers connected to a common bus.

Computer Instructions

The basic computer has three instruction code formats, as shown in Figure. Each format has 16 bits.

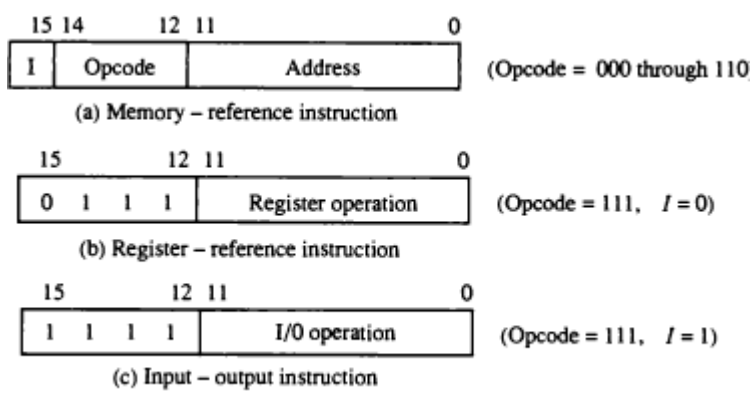


Figure 16 Basic computer instruction formats.

The type of instruction is recognized by the computer control from the four bits in positions 12through 15 of the instruction. If the three opcode bits in positions 12 through 14 are not equal to 111, the instruction is a memory-reference type and the bit in position 15 is taken as the addressing mode J. If the 3-bit opcode is equal to 111, control then inspects the bit in position 15. If this bit is 0, the instruction is a register-reference type. If the bit is 1, the instruction is an input-output type. Note that the bit in position 15 of the instruction code is designated by the symbol J but is not used as a mode bit when the operation code is equal to 111.
The instructions for the computer are listed in Table(10):

TABLE 10 Basic Computer Instructions

Symbol	Hexadecimal code $I = 0$	Hexadecimal code $I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear *AC*
CLE	7400		Clear *E*
CMA	7200		Complement *AC*
CME	7100		Complement *E*
CIR	7080		Circulate right *AC* and *E*
CIL	7040		Circulate left *AC* and *E*
INC	7020		Increment *AC*
SPA	7010		Skip next instruction if *AC* positive
SNA	7008		Skip next instruction if *AC* negative
SZA	7004		Skip next instruction if *AC* zero
SZE	7002		Skip next instruction if *E* is 0
HLT	7001		Halt computer
INP	F800		Input character to *AC*
OUT	F400		Output character from *AC*
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

The set of instructions are said to be complete if the computer includes a sufficient number of instructions in each of the following categories:

1. Arithmetic, logical, and shift instructions.
2. Instructions for moving information to and from memory and processor registers.
3. Instructions that check status information to provide decision making capabilities.
4. Input and output instructions.
5. The capability of stopping the computer

Instruction Cycle

A program residing in the memory unit of the computer consists of a sequence of instructions. The program is executed in the computer by going through a cycle for each instruction. In the basic computer each instruction cycle consists of the following phases:

1. Fetch an instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory if the instruction has an indirect address.
4. Execute the instruction.

This process continues indefinitely unless a HALT instruction is encountered. Initially, the program counter PC is loaded with the address of the first instruction in the program. The sequence counter SC is cleared to 0, providing a decoded timing signal T0. After each clock pulse, SC is incremented by one, so that the timing signals go through a sequence T0, T1, T2, and so on. The microoperations for the fetch and decode phases can be specified by the following register transfer statements.

$T0: AR \leftarrow PC$

$T1: IR \leftarrow [AR], PC = PC + 1$

$T2: D0, , , , D7 \leftarrow Decode\ IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$

It is necessary to use timing signal T1 to provide the following connections in the bus system.

1. Enable the read input of memory.
2. Place the content of memory onto the bus by making $S_2S_1S_0$ =111.
3. Transfer the content of the bus to IR by enabling the LD input of IR.
4. Increment PC by enabling the INR input of PC.

Memory - Reference Instructions

The table(11) lists the seven memory-reference instructions. The decoded output D, fori = 0,1, 2, 3, 4, 5, and 6 from the operation decoder that belongs effective address to each instruction is included in the table. The effective address of the instruction is in the address register AR and was placed there during timing signal T2 when I = 0, or during timing signal T3 when I=1. The symbolic description of each instruction is specified in the table in terms of register transfer notation. The actual execution of the instruction in the bus system will require a sequence of microoperations.

TABLE 11 Memory-Reference Instructions

Symbol	Operation decoder	Symbolic description	Means
AND	D_0	$AC \leftarrow AC \wedge M[AR]$	AND to AC
ADD	D_1	$AC \leftarrow AC + M[AR], \quad E \leftarrow C_{out}$	ADD to AC
LDA	D_2	$AC \leftarrow M[AR]$	LOAD to AC
STA	D_3	$M[AR] \leftarrow AC$	STORE AC
BUN	D_4	$PC \leftarrow AR$	Branch Unconditional
BSA	D_5	$M[AR] \leftarrow PC, \quad PC \leftarrow AR + 1$	Branch and save address
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1$, If $M[AR] + 1 = 0$ then $PC \leftarrow PC + 1$	Increment and skip if 0